

Báo cáo thực hành

Bài 13. Lập trình hợp ngữ với ESP-32-C3 – Mô phỏng bằng Wokwi

Assignment 1

1. Mã nguồn

```
.global init

# Định nghĩa các địa chỉ thanh ghi
.equ GPIO_ENABLE_REG, 0x60004020    # Thanh ghi cho phép xuất tín hiệu
.equ GPIO_OUT_WITS_REG, 0x60004008   # Thanh ghi thiết lập mức cao (High)
.equ IO_MUX_GPIO4_REG, 0x60004014   # Thanh ghi cấu hình chức năng chân GPIO4

.text
init:
    # --- Bước 1: Cấu hình hướng Output cho GPIO2, 3, 4 ---
    # Giá trị 0x1C (nhị phân: 11100) để bật bit 2, 3 và 4
    li a1, GPIO_ENABLE_REG
    li a2, 0x1C
    sw a2, 0(a1)

    # --- Bước 2: Cấu hình chức năng cho chân GPIO4 ---
    # Chân GPIO4 mặc định là SPI, cần chuyển sang chức năng GPIO (MCU_SEL = 1)
    # Giá trị 0x1000 tương ứng bit 12 được thiết lập là 1
    li a1, IO_MUX_GPIO4_REG
    li a2, 0x1000
    sw a2, 0(a1)

    # --- Bước 3: Xuất mức logic 1 để bật LED ---
    # Ghi giá trị 0x1C vào thanh ghi WITS để kéo các chân 2, 3, 4 lên mức cao
    li a1, GPIO_OUT_WITS_REG
    li a2, 0x1C
    sw a2, 0(a1)

    # Kết thúc chương trình
loop_forever:
    j loop_forever
```

2. Kết quả chạy

GPIO – 0	GPIO - 2
----------	----------



3. Giải thích

- Mặt nạ bit 0x1C: Đây là giá trị quan trọng nhất. Trong hệ nhị phân, 0x1C là 11100. Điều này có nghĩa là ta đang tác động cùng lúc vào bit thứ 2, 3 và 4 (tương ứng GPIO2, GPIO3, GPIO4).
- Thanh ghi GPIO_ENABLE_REG: Dùng để thông báo cho vi điều khiển rằng các chân này sẽ đóng vai trò là đầu ra.
- Thanh ghi IO_MUX_GPIO4_REG: Chân GPIO4 mặc định dùng cho giao thức SPI. Ta cần ghi giá trị 0x1000 (đặt trường MCU_SEL bằng 1) để ép nó hoạt động như một chân

GPIO thông thường. (GPIO2 và GPIO3 mặc định đã là GPIO nên không nhất thiết phải cấu hình IO_MUX).

- Thanh ghi GPIO_OUT_W1TS_REG: "W1TS" viết tắt của *Write 1 To Set*. Khi ta ghi bit 1 vào vị trí nào trong thanh ghi này, chân GPIO tương ứng sẽ xuất điện áp mức cao (3.3V), làm đèn LED sáng.

Assignment 2

1. Mã nguồn

```
.global init

# Định nghĩa các địa chỉ thanh ghi
.equ GPIO_ENABLE_REG, 0x60004020
.equ GPIO_OUT_W1TS_REG, 0x60004008 # Thanh ghi Set (Bật)
.equ GPIO_OUT_W1TC_REG, 0x6000400C # Thanh ghi Clear (Tắt)
.equ IO_MUX_GPIO4_REG, 0x60004014 # Cấu hình chân GPIO4

.text
init:
    # --- Bước 1: Cấu hình Output cho GPIO2, 3, 4 (Mặt nạ 0x1C = 11100 nhị phân) ---
    li a1, GPIO_ENABLE_REG
    li a2, 0x1C
    sw a2, 0(a1)

    # --- Bước 2: Chuyển GPIO4 sang chế độ GPIO (MCU_SEL = 1) ---
    li a1, IO_MUX_GPIO4_REG
    li a2, 0x1000
    sw a2, 0(a1)

main_loop:
    # --- Bước 3: Bật cả 3 LED ---
    li a1, GPIO_OUT_W1TS_REG
    li a2, 0x1C
    sw a2, 0(a1)
    call delay_asm

    # --- Bước 4: Tắt cả 3 LED ---
    li a1, GPIO_OUT_W1TC_REG
    li a2, 0x1C
    sw a2, 0(a1)
    call delay_asm

    j main_loop
```

--- Chương trình con tạo độ trễ ---

delay_asm:

li a3, 0

li a4, 8000000 # THAY ĐỔI TỐC ĐỘ: Tăng số này để nhảy chậm, giảm để nhảy nhanh

loop_delay:

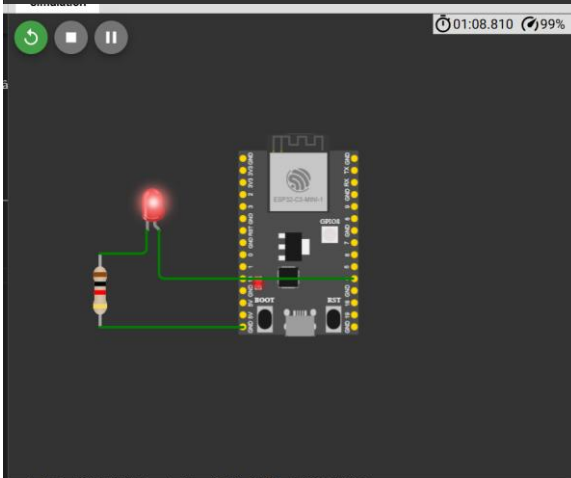
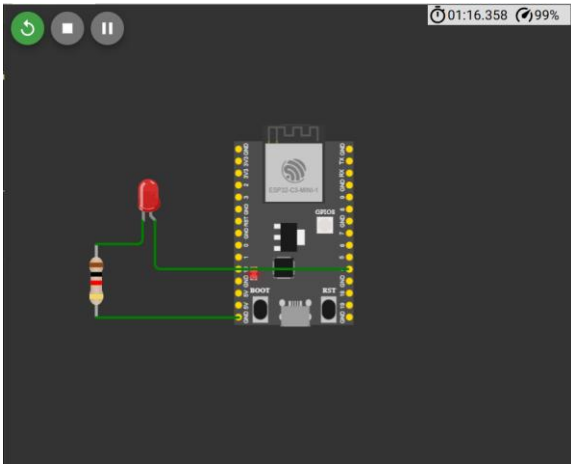
addi a3, a3, 1

blt a3, a4, loop_delay

ret

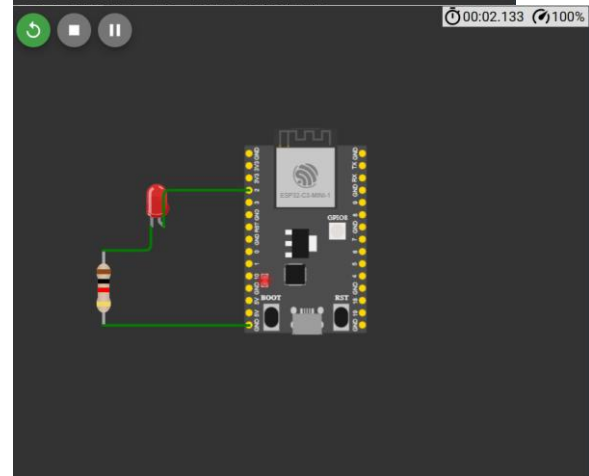
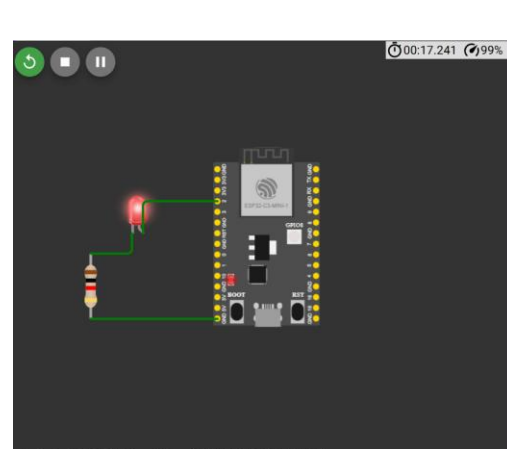
2. Kết quả chạy

GPIO – 4



GPIO – 3

GPIO – 2





3. Giải thích

- Sử dụng mặt nạ bit 0x1C: Thay vì chỉ sử dụng 0x01 (cho GPIO0), ta dùng 0x1C (nhị phân là 11100) để tác động đồng thời lên các chân số 2, 3 và 4.
- Cấu hình IO_MUX GPIO4: Tương tự Assignment 1, chân GPIO4 cần được thiết lập lại chức năng để có thể xuất tín hiệu Output ổn định.
- Điều khiển Bật/Tắt:
 - Ghi vào W1TS (Write 1 To Set) để đưa điện áp lên mức cao (Bật LED).
 - Ghi vào W1TC (Write 1 To Clear) để đưa điện áp về mức thấp (Tắt LED).
- Thay đổi thời gian nhấp nháy: Trong chương trình con `delay_asm`, giá trị của thanh ghi `a4` được thay đổi thành 8,000,000. Việc tăng giá trị này khiến vòng lặp đếm chạy lâu hơn, dẫn đến tốc độ nhấp nháy của đèn LED chậm lại.

Assignment 3

1. Mã nguồn

```

.global init
.equ GPIO_ENABLE_REG, 0x60004020
.equ GPIO_OUT_REG, 0x60004004
.equ IO_MUX_GPIO4_REG, 0x60009014
.equ IO_MUX_GPIO5_REG, 0x60009018
.equ IO_MUX_GPIO6_REG, 0x6000901C
.equ IO_MUX_GPIO7_REG, 0x60009020

.text
init:
    # 1. Cho phép xuất tín hiệu chân GPIO0 đến GPIO7
    li a1, GPIO_ENABLE_REG
    li a2, 0xFF
    sw a2, 0(a1)

    # 2. Chuyển chức năng sang GPIO cho các chân 4, 5, 6, 7 (thoát chế độ SPI)
    li a2, 0x1000
    li a1, IO_MUX_GPIO4_REG
    sw a2, 0(a1)
    li a1, IO_MUX_GPIO5_REG
    sw a2, 0(a1)
    li a1, IO_MUX_GPIO6_REG
    sw a2, 0(a1)
    li a1, IO_MUX_GPIO7_REG
    sw a2, 0(a1)

    # 3. HIỂN THỊ SỐ 0
    # Theo sơ đồ JSON: F=bit6, G=bit5, E=bit4, D=bit3, C=bit2, B=bit1, A=bit0
    # Số 0 cần sáng: A,B,C,D,E,F (tắt G) -> Nhị phân: 01011111 -> Hex: 0x5F
    li a1, GPIO_OUT_REG
    li a2, 0x5F
    sw a2, 0(a1)

loop:
    j loop

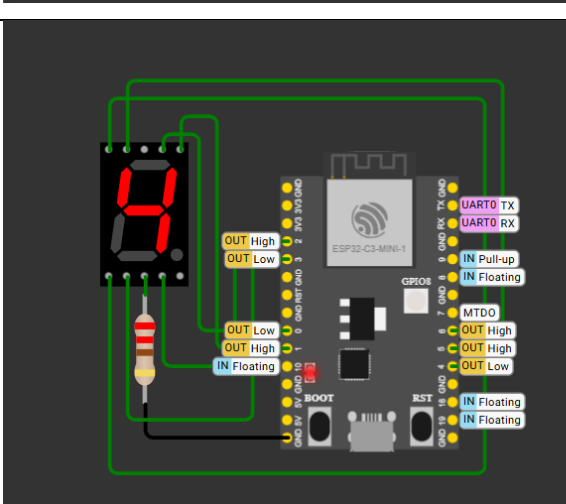
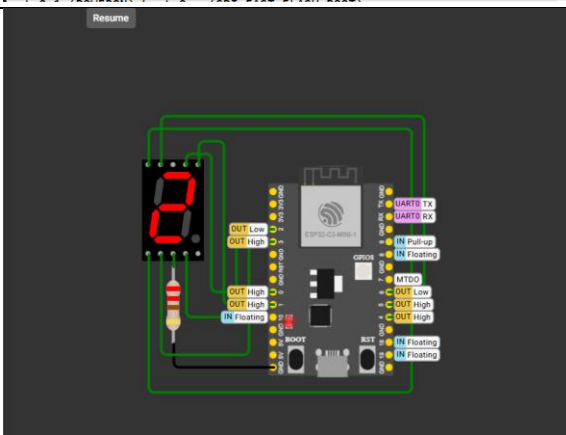
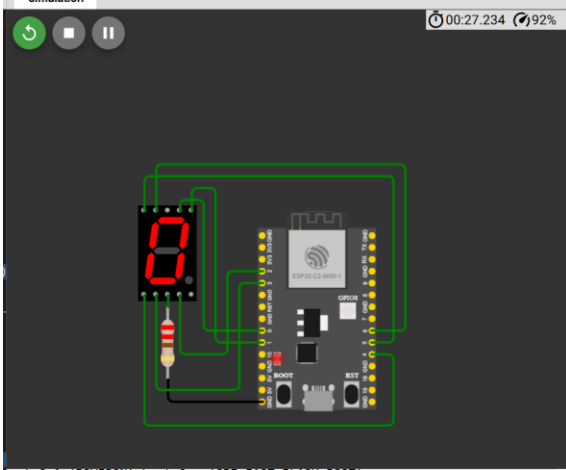
```

2. Kết quả chạy

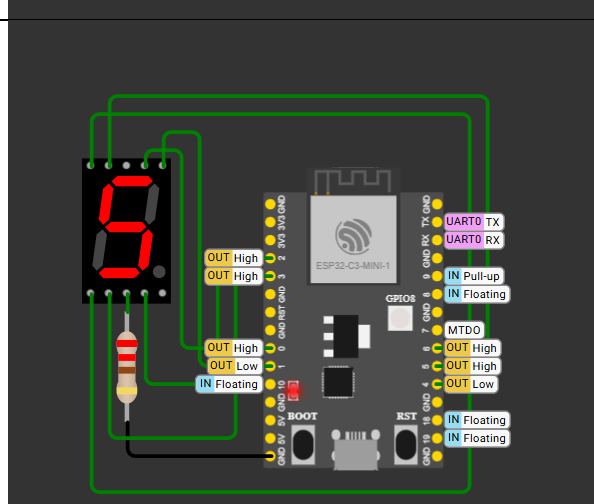
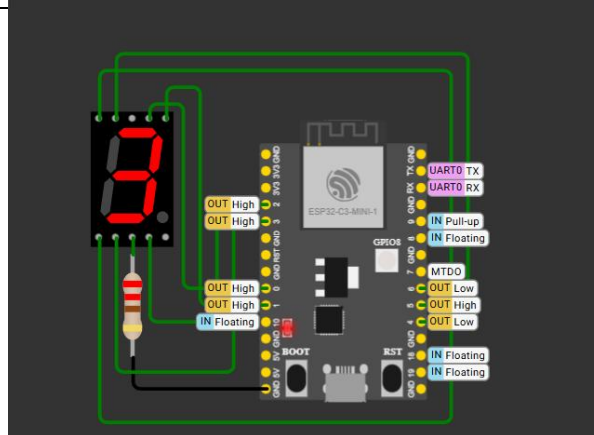
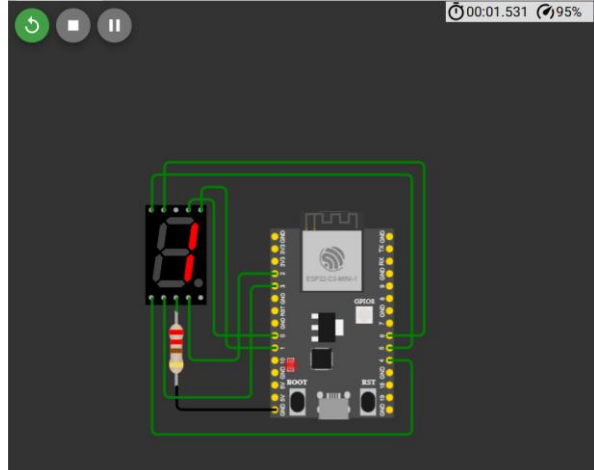
Chữ số	Các đoạn cần sáng	Mã nhị phân (F-G-E-D-C-B-A)	Mã Hex (Nhập vào a2)
0	A, B, C, D, E, f	1011111	0x5F
1	B, C	0000110	0x06
2	A, B, G, E, D	0111011	0x3B
3	A, B, G, C, D	0101111	0x2F
4	F, G, B, C	1100110	0x66
5	A, F, G, C, D	1101101	0x6D

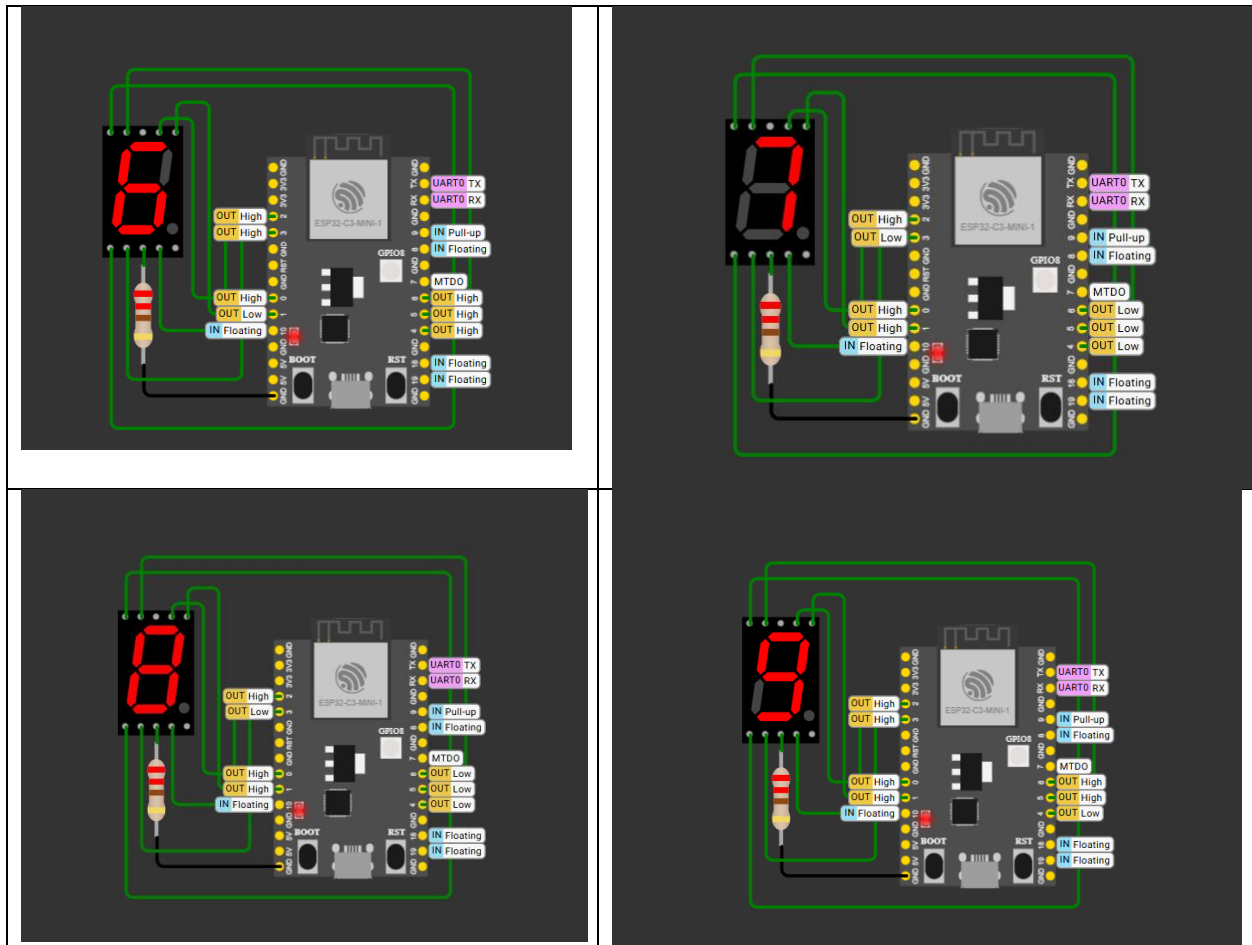
6	A, F, E, D, C, G	1111101	0x7D
7	A, B, C	0000111	0x07
8	Tất cả (A,B,C,D,E,F,G)	1111111	0x7F
9	A, B, C, D, F, G	1101111	0x6F

A2 = 0xFF



A2 = 0x006





Assignment 4

1. Mã nguồn

```
.global init
```

```
# Định nghĩa địa chỉ thanh ghi
```

```
.eqv GPIO_OUT_WITS_REG, 0x60004008 # Thanh ghi bật LED (Set)
```

```
.eqv GPIO_OUT_WITC_REG, 0x6000400C # Thanh ghi tắt LED (Clear)
```

```
.eqv GPIO_ENABLE_REG, 0x60004020 # Cho phép xuất tín hiệu (Output)
```

```
.eqv GPIO_IN_REG, 0x6000403C # Đọc trạng thái chân (Input)
```

```
.eqv IO_MUX_GPIO0_REG, 0x60009004 # Cấu hình chân GPIO0
```

```
.text
```

```
init:
```

```
    # 1. Thiết lập GPIO1 là đầu ra (Output) cho LED
```

```
    li a1, GPIO_ENABLE_REG
```

```
    li a2, 0x02 # Bit 1 ứng với GPIO1
```

```
    sw a2, 0(a1)
```



```

# 2. Thiết lập GPIO0 là đầu vào (Input) để đọc công tắc
# Phải bật bit FUN_IE (bit 9 - giá trị 0x200) để chân có thể nhận tín hiệu
li a1, IO_MUX_GPIO0_REG
lw a2, 0(a1)          # Đọc giá trị cấu hình hiện tại
li t0, 0x200          # Giá trị mặt nạ bit cho FUN_IE
or a2, a2, t0         # Dùng lệnh OR để bật bit 9 mà không mất các bit khác
sw a2, 0(a1)

```

loop:

```

# 3. Đọc toàn bộ trạng thái các chân GPIO
li a1, GPIO_IN_REG
lw a2, 0(a1)

```

```

# 4. Kiểm tra trạng thái của chân GPIO0 (bit 0)
# Mặt nạ 0x01 để lấy đúng bit của GPIO0
andi a3, a2, 0x01

```

```

# Nếu GPIO0 = 0 (Công tắc gạt về GND) -> Nhảy tới tắt LED
beq a3, zero, led_off

```

led_on:

```

# Bật LED tại GPIO1 (Ghi mức 1 vào bit 1 của W1TS)
li a1, GPIO_OUT_W1TS_REG
li a2, 0x02
sw a2, 0(a1)
j next

```

led_off:

```

# Tắt LED tại GPIO1 (Ghi mức 1 vào bit 1 của W1TC)
li a1, GPIO_OUT_W1TC_REG
li a2, 0x02
sw a2, 0(a1)

```

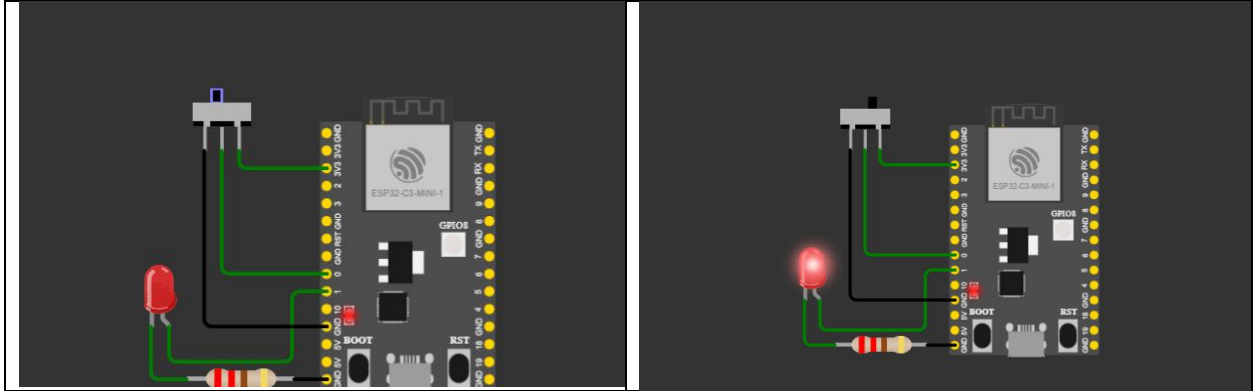
next:

```

j loop          # Lặp lại liên tục

```

2. Kết quả chạy



Assignment 5

1. Mã nguồn:

```
.global init

.equ GPIO_ENABLE_REG, 0x60004020
.equ GPIO_OUT_REG, 0x60004004

.equ IO_MUX_GPIO4_REG, 0x60009014
.equ IO_MUX_GPIO5_REG, 0x60009018
.equ IO_MUX_GPIO6_REG, 0x6000901C

.text

init:

# bật output GPIO0->GPIO7
li a1, GPIO_ENABLE_REG
li a2, 0xFF
sw a2, 0(a1)

# chuyển GPIO4-6 sang chức năng GPIO
li a2, 0x1000

li a1, IO_MUX_GPIO4_REG
sw a2, 0(a1)

li a1, IO_MUX_GPIO5_REG
sw a2, 0(a1)

li a1, IO_MUX_GPIO6_REG
sw a2, 0(a1)
```

main:

0

li a1, GPIO_OUT_REG

li a2, 0xC0

sw a2, 0(a1)

call delay

1

li a2, 0xF9

sw a2, 0(a1)

call delay

2

li a2, 0xA4

sw a2, 0(a1)

call delay

3

li a2, 0xB0

sw a2, 0(a1)

call delay

4

li a2, 0x99

sw a2, 0(a1)

call delay

5

li a2, 0x92

sw a2, 0(a1)

call delay

6

li a2, 0x82

sw a2, 0(a1)

call delay

7

li a2, 0xF8

sw a2, 0(a1)

call delay

8

li a2, 0x80

sw a2, 0(a1)

call delay

9

li a2, 0x90

sw a2, 0(a1)

call delay

j main

delay:

li t0, 0

li t1, 3000000

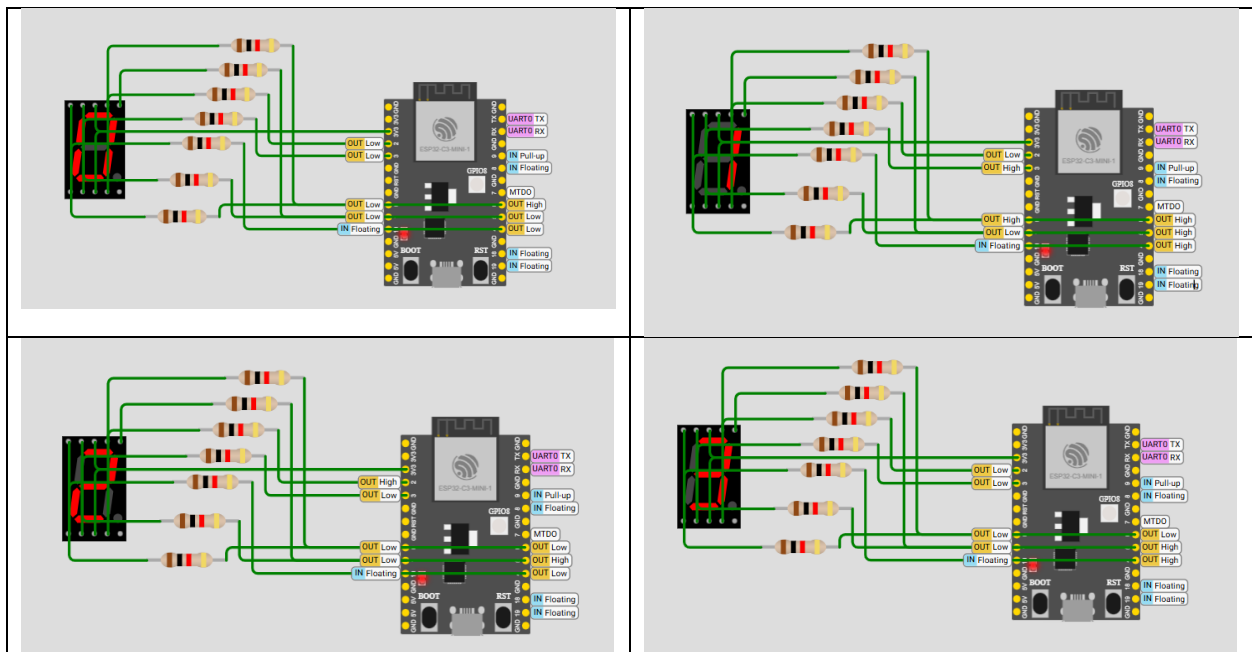
loop:

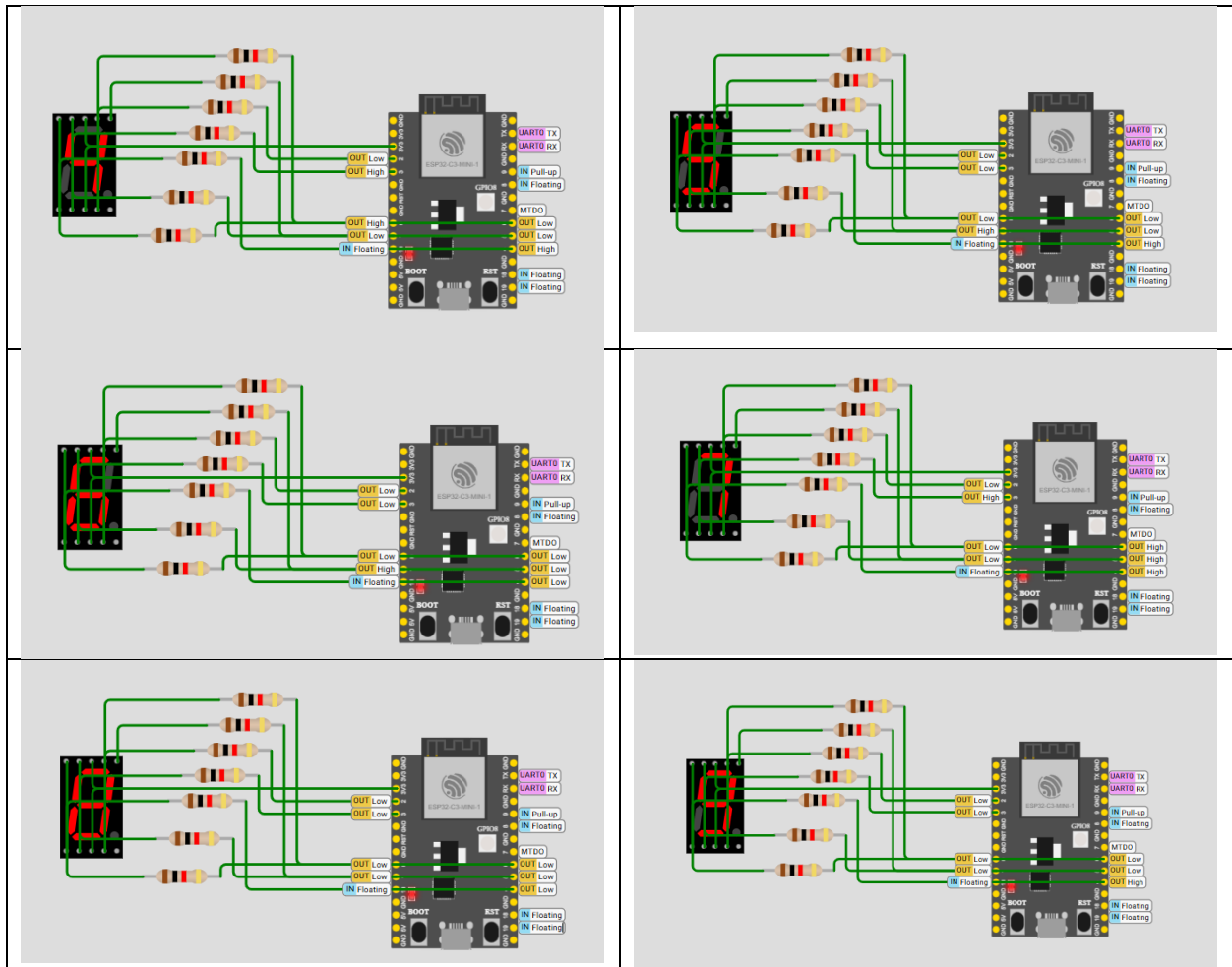
addi t0, t0, 1

blt t0, t1, loop

ret

2. Kết quả:





3. Giải thích

Cách hoạt động:

1. **Cấu hình GPIO0–GPIO6 là chân Output** để điều khiển các đoạn a→g của LED.
2. **Xuất mã nhị phân (mã hex)** tương ứng với từng số vào thanh ghi GPIO_OUT_REG.
3. LED sẽ bật/tắt các đoạn tương ứng để hiện số.
4. Gọi hàm **delay** để giữ số hiện đủ lâu.
5. Sau delay → xuất mã số tiếp theo.
6. Sau số 9 → quay lại số 0 → lặp vô hạn.